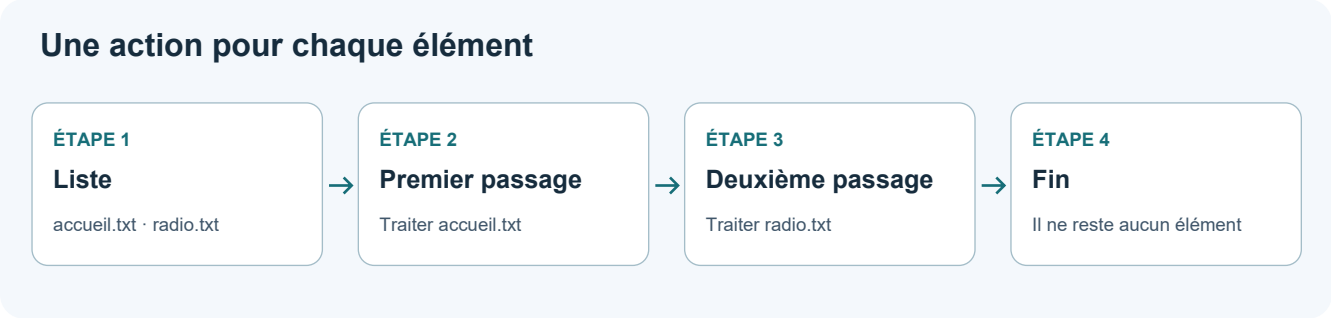
Chapitre 05 — Les boucles

Objectif Répéter une action sur une liste ou un groupe de fichiers, sans recopier les mêmes commandes.

1. Une action, plusieurs éléments



La même action pour chaque fichier

Une boucle `for` prend les éléments **un par un**. À chaque passage, une variable contient l'élément courant. Voici la forme générale, à lire comme un modèle :

```
for variable in element1 element2 element3; do
    # commandes utilisant "$variable"
done
```

Appliquée à une liste de prénoms :

```
for personne in Mila Noe Vega; do
    echo "Bonjour $personne"
done
```

Résultat : trois lignes, une par personne. `do` ouvre le corps de la boucle ; `done` le ferme.

Un élément peut contenir des espaces :

```
for poste in "salle radio" "poste de pilotage"; do
    echo "Controle : $poste"
done
```

Les guillemets conservent ici **deux éléments**, pas six mots.

2. Répéter un nombre de fois

Une liste, ou une plage

Les éléments peuvent être des nombres :

```
for numero in 1 2 3; do
    echo "Test numero $numero"
done
```

Au-delà de quelques valeurs, écris une **plage** plutôt que la liste complète :

```
for numero in {1..10}; do
    echo "Test numero $numero"
done
```

`{1..10}` est remplacé par Bash avant l'exécution : la boucle reçoit bien dix éléments. Tu as déjà croisé cette écriture dans les démos du chapitre 01 — c'est une facilité de Bash que `sh` ne connaît pas.

> Une plage n'accepte pas de variable Bash remplace `{1..10}` **avant** de lire les variables. Avec une borne variable, le remplacement n'a donc pas lieu :

```
max=5
for numero in {1..$max}; do echo "$numero"; done
```

```
{1..5}
```

> Un seul passage, avec le texte littéral. Pour une borne variable, utilise une boucle `while` comme au §4.

Compter les passages

Une boucle peut aussi tenir le compte de ce qu'elle a traité :

```
compteur=0
for objet in radio casque batterie; do
    compteur=$((compteur + 1))
    echo "$compteur : $objet"
done
echo "Objets controles : $compteur"
```

On initialise le compteur **avant** la boucle, on augmente sa valeur **dans** la boucle, on le lit **après**. Le résultat final est 3.

3. Parcourir des fichiers avec un motif

C'est l'usage le plus fréquent de `for` : agir sur tous les fichiers qui portent un certain nom.

Prépare quelques données :

```
mkdir -p ~/atelier-bash/boucles/messages
cd ~/atelier-bash/boucles
echo 'Bienvenue' > messages/accueil.txt
echo 'Radio active' > "messages/radio du jour.txt"
echo 'Image fictive' > messages/image.png
```

```
for fichier in messages/*.txt; do
    if [[ -f "$fichier" ]]; then
        echo "Lecture de $fichier"
        cat "$fichier"
    fi
done
```

`*.txt` est un **motif de noms** : Bash le remplace par la liste des noms qui correspondent, avant que la boucle démarre. Le fichier `image.png` n'est pas sélectionné. C'est ce mécanisme qui rend `for` utile au quotidien — tu n'as pas à connaître les noms à l'avance.

Le test `-f` évite de traiter un dossier. Il gère aussi le cas où **aucun** nom ne correspond : avec les réglages habituels de Bash, le motif reste alors un texte littéral, qui n'est pas un fichier.

```
for fichier in messages/*.csv; do echo "[$fichier]"; done
```

```
[messages/*.csv]
```

Un seul passage, avec le motif non remplacé : sans le test `-f`, la boucle essaierait de lire un fichier nommé `*.csv`.

> La bonne place des guillemets Écris `"$dossier"/*.txt` si le dossier vient d'une variable. La variable est protégée ; le `*` reste disponible pour la recherche. Ne fais pas une boucle sur le résultat de `ls` : les noms contenant des espaces seraient difficiles à traiter correctement.

4. Une boucle while répète tant que le test réussit

Là où `for` parcourt une liste connue d'avance, `while` répète tant qu'une condition reste vraie. Forme générale :

```
while test; do
    # commandes repetees tant que le test reussit
    # l'une d'elles doit faire evoluer le test
done
```

```
compteur=3
while [[ "$compteur" -gt 0 ]]; do
    echo "Depart dans $compteur"
    compteur=$((compteur - 1))
done
echo 'Decollage'
```

La valeur descend de 3 à 1. Quand elle vaut 0, le test échoue et la boucle se termine.

La ligne `compteur=$((compteur - 1))` est celle qui fait évoluer le test. Si tu l'oublies, la condition ne changera jamais : **Ctrl+C** permet d'interrompre une boucle sans fin.

5. Lire un fichier ligne par ligne

```
printf 'Mila Vega\nNoe\n' > equipe.txt
while IFS= read -r personne; do
    echo "Equipier : $personne"
done < equipe.txt
```

Nous utilisons ici `printf` plutôt qu'`echo` pour écrire **deux lignes en une commande** : c'est le cas de figure du chapitre 02 où `printf` garde l'avantage.

- `read` lit une ligne à chaque passage, et la boucle s'arrête quand il n'y en a plus.
- `< equipe.txt` fournit le fichier comme entrée à la boucle.
- `IFS=` et `-r` protègent le contenu de la ligne.

> Pourquoi `IFS=` et `-r` ? Sur des lignes ordinaires, ils ne changent rien — c'est pour cela qu'on les oublie. Leur effet n'apparaît que sur un contenu moins sage :

```
printf '%s\n' '   Mila Vega   ' 'C:\Orion\radio' > brut.txt
```

Lecture	Première ligne	Seconde ligne
`while IFS= read -r ligne`	`[ Mila Vega ]`	`[C:\Orion\radio]`
`while read ligne`	`[Mila Vega]`	`[C:Orionradio]`

Sans `IFS=`, les espaces de début et de fin sont retirés. Sans `-r`, les antislashs sont pris pour des caractères d'échappement et disparaissent. Les données réelles — noms de fichiers, chemins, champs recopiés — contiennent ce genre de surprises.

Retiens `while IFS= read -r ligne; do ... done < fichier` comme un bloc : c'est la façon correcte de lire un fichier ligne par ligne.

Ici, les lignes se terminent toutes par un retour à la ligne, créé avec `\n`. Pour traiter aussi une dernière ligne sans ce retour, on peut compléter la condition ; ce cas est proposé en bonus du TP.

6. Passer un élément ou arrêter la boucle

`continue` passe directement à l'élément suivant. `break` quitte la boucle.

```
for message in INFO IGNORER ALERTE FIN; do
    if [[ "$message" == "IGNORER" ]]; then
        continue
    fi
    if [[ "$message" == "FIN" ]]; then
        break
    fi
    echo "Message : $message"
done
```

Résultat : `INFO` puis `ALERTE`. `IGNORER` est sauté, et `FIN` interrompt le parcours avant `echo`. Commence par une boucle simple ; ajoute ces instructions seulement si elles rendent le parcours plus clair.

7. Exemple complet : copier les messages

Dans le dossier `boucles`, crée `copier.sh` :

fichier: copier.sh

```
#!/usr/bin/env bash

if [[ "$#" -gt 1 ]]; then
    echo "Usage : $0 [dossier]" >&2
    exit 2
fi

source="${1:-messages}"
if [[ ! -d "$source" ]]; then
    echo "Dossier absent : $source" >&2
    exit 2
fi
```

```
destination="$source/copies"
if ! mkdir -p -- "$destination"; then
    echo 'Impossible de creer la destination' >&2
    exit 1
fi

compteur=0
for fichier in "$source"/*.txt; do
    if [[ ! -f "$fichier" ]]; then
        continue
    fi
    if ! cp -- "$fichier" "$destination/"; then
        echo "Copie impossible : $fichier" >&2
        exit 1
    fi
    compteur=$((compteur + 1))
done

echo "Copies realisees : $compteur"
```

Ce script réunit tout le chapitre : une garde sur les arguments, un motif, un `continue`, un compteur, et un code de sortie par type d'erreur.

`--` marque la fin des options pour `mkdir` et `cp`. Un nom commençant par `-` ne sera donc pas pris pour une option. `cp` copie les fichiers sans supprimer les originaux.

```
bash -n copier.sh
bash copier.sh messages
ls messages/copies
```

Deux fichiers doivent être copiés. Ce petit outil remplace les copies de même nom lors d'un nouveau lancement : ce n'est pas un système de sauvegarde avec historique.

8. Écrire une fonction

Une fonction donne un nom à un groupe de commandes. Tu peux l'appeler plusieurs fois, ce qui évite de répéter les mêmes lignes dans une boucle ou dans un script :

```
saluer() {
    local personne="${1:-equipier}"
    echo "Bienvenue $personne"
}

saluer Mila
saluer Vega
saluer
```

Résultat : `Bienvenue Mila`, `Bienvenue Vega`, puis `Bienvenue equipier`.

- Les arguments de la fonction se lisent avec `$1`, `$2`, comme ceux d'un script.
- `local` réserve la variable à cette fonction : elle n'écrase pas une variable du même nom ailleurs dans le script.
- La fonction doit être **définie avant** d'être appelée : place-la en haut du fichier.

Les TP de synthèse et le TP final utilisent une fonction pour écrire les titres d'un rapport : ce n'est pas une notion facultative.

À retenir

- `for variable in liste; do ... done` parcourt des éléments connus ; `while test; do ... done` répète tant qu'une condition réussit.
- La plage `{1..10}` évite d'écrire une longue liste de nombres, mais n'accepte pas de borne variable.
- Un compteur s'initialise **avant** la boucle et s'incrémente **dans** la boucle avec `$((compteur + 1))`.
- Le motif `*.txt` est remplacé par les noms correspondants avant le démarrage de la boucle. S'il ne correspond à rien, il reste littéral : garde un test `-f`.
- `while IFS= read -r ligne; do ... done < fichier` est la façon correcte de lire un fichier ligne par ligne.
- `continue` saute un élément, `break` arrête la boucle.
- Une fonction nomme un groupe de commandes ; `local` y réserve une variable.
- Protège les noms avec `"$variable"`, et pense à tester une liste vide, un nom composé et un dossier absent.

À toi : que se passe-t-il si le motif `*.txt` ne correspond à aucun fichier ? Pourquoi `IFS=` et `-r` ne changent-ils rien sur `Mila Vega`, mais tout sur `C:\Orion\radio` ?

Passer au TP